

以太坊（2）——共识机制与挖矿

共识机制

ETH采用的是基于GHOST协议的共识机制

"GHOST" (Greedy Heaviest-Observed Sub-Tree) 共识机制，它是以太坊使用的一种改进的区块链共识算法。GHOST共识机制旨在提高链的安全性和效率，通过考虑非主链区块的贡献，减少网络中的分支和拜占庭错误，从而增加整体网络的可靠性。

新的共识机制

原因

ETH的区块产生时间短，更容易出现分支

1. 新的共识机制着力于解决挖出分支链该怎么办的问题
2. 新的共识机制解决了 **挖矿奖励与算力不成线性正比** 和 **使分支回到最长合法链** 的两个问题

解决方法

对于挖出区块属于的非最长合法链的矿工予以补偿

为什么？

在ETH体系中，依旧有矿池的存在。假设按照BTC的处理方法

一个矿池挖出了一个区块，那么他肯定会沿着这个区块继续挖下去，而其他矿工的算力是分散的，而且更倾向于沿着矿池挖掘的最长合法链挖矿。这就导致了散户矿工拿到的收益与矿池拿到的收益和他们付出的算力不成正比，出现mining centralization的现象，矿池占到更大的便宜。

共识机制的演化

第一版

协议

1. 每个区块可以包含uncle block,
2. 被包含的uncle block获得出块奖励的7/8
3. 包含uncle block的新区块额外获得出块奖励的1/32
4. 对于uncle block，只判断它的哈希值是否合法，不管它的内容

优点

1. 补偿了挖出分支区块的矿工
2. 使得区块链分叉后的更快合并

存在问题

1. 有的矿工去到很早的节点（几千几万个之前）挖分支区块，由于那时候挖矿难度低，他能获得大量区块，这时候他在主链上把这些区块当成uncle block包含进去，获得大量的以太币。
2. 某些矿池为了压倒对手，故意不承认uncle block。损己一分，伤敌28倍

第二版

协议改进

1. 规定uncle block获得的奖励为1-n/8。七代以内有共同祖先的区块才能算是uncle block
2. 每个区块最多可以包含两个uncle block

改进意义

1. 鼓励矿工尽早解决分支问题
2. 小矿工可以尽早转入最长合法链，将自己的区块认作uncle block

区块实例

Height:	< Prev 5695161 Next >	Source:Etherscan.io
TimeStamp:	19 mins ago (May-29-2018 04:45:25 AM +UTC)	
Transactions:	89 transactions and 3 contract internal transactions in this block	
Hash:	0x76df197457effdbb736480393c70a016fe3bbdbfe619d16640cb665d748dcdf	
Parent Hash:	0xbd3ecbcf5527bb6de899912cb86eadc762c86832c713bef3910b0ec7184e0f7a	
Sha3Uncles:	0xde903bc6ba5e5ca6155d936f882a92a653f3b0a60a346f0474fc56e61340ea9	
Mined By:	0xea674fdd6714fd979de3edf0f56aa9716b898ec8 (Ethermine) in 20 secs	
Difficulty:	3,184,956,261,907,541	
Total Difficulty:	4,459,340,439,119,129,119,115	
Size:	18032 bytes	
Gas Used:	7,967,412 (99.74%)	
Gas Limit:	7,988,337	
Nonce:	0xd280930018199336	
Block Reward:	3.260603241218831558 Ether (3 + 0.166853241218831558 + 0.09375)	
Uncles Reward:	2.25 Ether (1 Uncle at Position 0)	
Extra Data:	ethermine-aws-us1-1 (Hex:0x65746865726d696e652d6177732d7573312d31)	

Height:	< Prev 5695150 Next >	Source:Etherscan.io
TimeStamp:	23 mins ago (May-29-2018 04:41:51 AM +UTC)	
Transactions:	160 transactions and 9 contract internal transactions in this block	
Hash:	0x92174a45e568b53e7aa0bdee81c73e7de9d214827546d11532f0c023889f4ee6	
Parent Hash:	0x3335cadce8ad3842351f0223fd7b9e5e2d1f46f0dca4d7d2464c00750a33fd1e	
Sha3Uncles:	0xabfb2427e51f6879e15b13c1aa9d327ec748fe99637628596fdc9f1b3ed52e4c	
Mined By:	0xea674fdd6714fd979de3edf0f56aa9716b898ec8 (Ethermine) in 14 secs	
Difficulty:	3,189,637,521,586,694	
Total Difficulty:	4,459,305,357,839,994,234,039	
Size:	27993 bytes	
Gas Used:	7,994,188 (99.93%)	
Gas Limit:	8,000,029	
Nonce:	0xe1a977700b02217f	
Block Reward:	3.31510552614492296 Ether (3 + 0.12760552614492296 + 0.1875)	
Uncles Reward:	4.875 Ether (2 Uncles at Position 0, Position 1)	
Extra Data:	ethermine-eu8 (Hex:0x65746865726d696e652d657538)	

引入1个uncle的reward是 $3 \times \frac{1}{32} = 0.09375$

引入2个uncle的reward是 $2 \times 3 \times \frac{1}{32} = 0.1875$

共识机制补充

局限

共识机制无法改变之前讲到的state fork（软/硬分叉）

BTC和ETH的收入

BTC

静态收入：挖矿奖励

动态收入：交易手续费

静态收入逐渐降低，是为了营造比特币“数字黄金”的稀缺性

ETH

静态收入：挖出最长合法链区块奖励 挖出分支区块的补偿 认叔奖励

动态收入：gas

静态收入基本不会降低，保证了gas费不会成为主要收入，进而保证了对于矿工的补偿机制，可以理解
为，以太坊是汽油（不会消失的消耗品）

另类uncle block

对于一个长分支链，每个区块都不会获得分支奖励

因为如果奖励的话会降低分叉攻击的成本。

挖矿

BTC挖矿的问题

挖矿设备的专业化，降低了去中心化的程度

one CPU, one Vote

因此，此后的加密货币要实现的是ASIC resistance

ASIC resistance可以先看一下BTC部分的挖矿

解决方案

1. 内存，使得挖矿设备更接近通用计算机
2. 权益证明，盘外招？
3. 预挖矿，使得开发者拥有大量资产

方案一

设计需要大量内存的mining puzzle

这种mining puzzle的设计参考了莱特币

LiteCoin（莱特币）

1. Litecoin使用的是 scrypt mining puzzle
2. scrypt mining puzzle是一种需要大量内存的哈希算法
3. 设计原理：
 1. 创建一个数组，使用一个seed，计算出的值填在第一个位置
 2. 之后，第一个位置的数取哈希，放在第二个的位置，以此类推往后放，直到将整个数组填满
 3. 在求解mining_puzzle的时候，先取出一个指定位置的存储结果，将这个结果进行一定的运算得到下一个位置；取出“下一个位置”的值，按上述的方法以此类推，最终得到结果
4. 这样做的好处：使得求解过程变成memory_hord，如果你不用这个数组，计算难度就会大幅度上升（不存储，只能一次一次算到该位置的哈希）；如果你使用的话，对内存要求就会很高，实现了ASIC_resistance
5. ASIC_resistance设计的核心思想是：**增加计算过程对于内存的访问次数**，使得这个过程跟贴近于普通计算机对于计算资源的配备
6. 方案劣势：对于轻节点而言，也是memory_hard,因此在实际LiteCoin的使用中，最大的数组为128K。这也使得它对于ASIC的抵制性不高

ETH的挖矿设计

ETH挖矿使用了两个数据集小的是16M的 cache，大的是1G的dataset (DAG)

轻节点只用保存cache，矿工需要保存DAG

数据集

两个数据集的内存大小都在随时间上涨，适应计算机与日俱增的内存容量

cache

生成方式与莱特币数组生成的方式类似 (seed在cache中)

DAG

1. 根据cache，取出某个位置的数据，按照莱特币的方法，迭代256次，放入DAG数组的第一个位置，之后以同样的方法放在DAG的第二个位置，以此类推形成完整的DAG数据集
2. 求解mining_puzzle的方法是，依照nonce和BlockHeader的数据，映射到DAG的某一个位置；取出这个位置和相邻下一个位置的数据，计算哈希映射到之后的位置，以此类推循环64次计算出区块哈希
3. 验证区块哈希是否符合target要求

实现代码

ETHASH伪代码

生成cache

```
def mkcache(cache_size, seed):  
    o = [hash(seed)]  
    for i in range(1, cache_size):  
        o.append(hash(o[-1]))  
    return o
```

1. 这个函数是通过seed计算出来cache的伪代码(cache中每个元素都是64字节的hash值)。
2. 伪代码略去了原来代码中对cache元素进一步的处理，只展示原理，即cache中元素按序生成，每个元素产生时与上一个元素相关。
3. 每隔30000个块会重新生成seed(对原来的seed求哈希值)，并且利用新的seed生成新的cache。
4. cache的初始大小为16M，每隔30000个块重新生成时增大初始大小的1/128(128K)。

生成DAG

```
//生成DAG中的一个元素  
def calc_dataset_item(cache, i):  
    cache_size = cache.size  
    mix = hash(cache[i % cache_size] ^ i)  
    for j in range(256):  
        cache_index = get_int_from_item(mix)  
        mix = make_item(mix, cache[cache_index % cache_size])  
    return hash(mix)  
//多次调用生成整个DAG  
def calc_dataset(full_size, cache):  
    return [calc_dataset_item(cache, i) for i in range(full_size)]
```

1. 这是通过cache来生成dataset中第 i 个元素的伪代码。
2. 这个dataset叫作DAG,初始大小为1G, 每30000个块更新, 同时增大初始大小的1/128 (8M)
3. 先通过cache中的第i%cache_size个元素生成初始的mix, 因为两个不同的dataset元素可能对应同一个cache中的元素, 为了保证每个初始的mix都不同, (i也参与哈希计算)
4. 随后循环256次, 每次通过get_int_from_item来根据当前的mix值**求得一个要访问的cache元素的下标**, 用这个cache元素和mix通过make_item**求得新的mix值**。(由于初始的mix值都不同, 所以访问cache的序列也都是不同的)
5. 迭代256次最终返回mix的哈希值, 得到第 i 个dataset中的元素。
6. 多次调用这个函数, 就可以得到完整的dataset。

挖矿与验证

```
//全节点挖矿
////挖出一次hash（取名为最终哈希）
def hashimoto_full(header, nonce, full_size, dataset):
    mix = hash(header, nonce)
    for i in range(64):
        dataset_index= get_int_from_item(mix) % full_size
        mix = make_item(mix,dataset[dataset_index])
        mix = make_item(mix,dataset[dataset_index + 1])
    return hash(mix)
////挖出hash的主循环函数
def mine(full_size, dataset, header,target):
    nonce =random.randint(0, 2**64)
    while hashimoto_full(header, nonce, full_size, dataset)>target:
        nonce =(nonce +1)%2**64
    return nonce
//轻节点验证
def hashimoto_light(header, nonce, full_size, cache):
    mix = hash(header,nonce)
    for i in range(64):
        dataset_index = get_int_from_item(mix) % full_size
        mix = make_item(mix, calc_dataset_item(cache, dataset_index))
        mix = make_item(mix, calc_dataset_item(cache, dataset_index + 1))
    return hash(mix)
```

挖矿一次函数

1. 使用块头, 使得轻节点存储占据内存小, 方便验证。full_size是数据集的全部元素个数
2. 首先根据块头的信息和nonce算出初始的哈希值。经过64次循环, 每次循环读取大数据集中两个相邻的数（读取的位置由当前哈希值计算出）, 最后返回一个哈希值

挖出hash主循环函数

将调用一次上面挖矿函数返回的“最终哈希”与target进行比较, 如果不满足, 再调用挖矿函数

验证函数

1. 这里的nonce是发布的区块块头里的nonce; 使用cache验证, 但是这里的full_size指的还是DAG里面的元素个数。
2. 由于轻节点没有存储DAG, 因此需要从cache中重新生成, 其他操作的和上述生成一次最终hash类似
3. 验证的操作分成两步, 我们和BTC类比

1. **验证哈希来源是否合法**：BTC验证nonce值是不是生成hash的值，只需要结合块头进行一次哈希计算；而ETH的验证就是走一遍上面生成“最终哈希”的过程，看看生成的hash是不是公布的hash。
2. **验证哈希是否符合要求（区块是否合法）**：二者都验证了公布的hash是不是满足target要求

效果

对于轻节点而言，验证的时候只用进行一次“最终哈希的计算”，因此可以逐次计算1G的DAG，但是对于矿工而言。如果存储DAG，每次试“最终哈希”的时候都可以使用，如果不存储，每次都要算，计算量大大提升。

实际上，ETH使用的挖矿算法ethash，到目前为止有效的将挖矿手段限制在了GPU的水平，取得了很好的ASIC_resistance的效果

方案二

使用权益证明（PoS）替代工作量证明（Pow）

权益证明

在权益证明中，区块的产生不再依赖于计算复杂的数学题，而是通过持有和抵押一定数量的以太币（ETH）来获得网络的投票权。持有更多以太币的节点将有更高的几率被选为下一个出块节点，并且会因此获得相应的奖励。

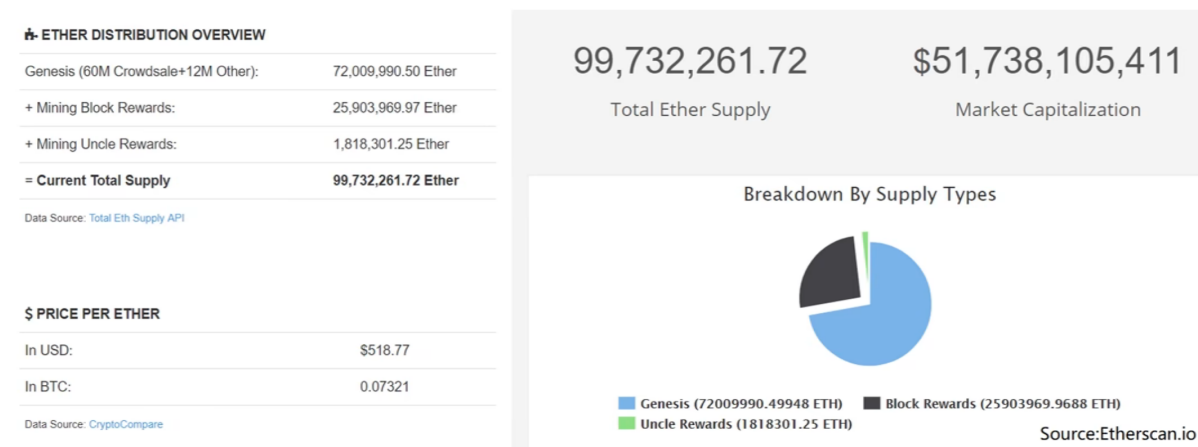
吓唬大家的手段？

之后会讲

方案三

预挖矿（Pre-mining）：发行的一部分货币留给以太坊的开发者

预售（Pre-sale）：将一部分Pre-mining货币换成资产（现实），用于以太坊的开发工作。如果投资者看好这种货币，可以在Pre-sale阶段购买，等到货币成熟（升值）之后获得收益。



挖矿挖的再努力，赢在起跑线上最重要！（dog）

关于安全性的另一种想法

使用ASIC挖矿有利于安全性提高

1. **增加了攻击的代价**ASIC芯片具有专一性，研发周期长，研发投入高的特点。因此，购买大量矿机的矿池，拥有大量的该货币资产。他们会尽量维持区块链的安全性，因为如果安全性降低（被攻击），该货币就会贬值。

1. 挖矿收入和现有资产缩水
2. 购入的矿机由于只能挖这一条链，所以现有的矿机化为废品
2. 如果通用计算机可以挖矿，那么一些不法分子可能通过租借云计算资源的方式进行攻击，攻击的硬件成本大大降低，区块链安全性下降

至于哪种说法对？具体情况具体分析

挖矿难度的调整

调整算法

基础调整

区块难度 $D(H)$

$$D(H) \equiv \begin{cases} D_0 & \text{if } H_i = 0 \\ \max(D_0, P(H)_{H_d} + x \times \varsigma_2) + \epsilon & \text{otherwise} \end{cases}$$

where:

(42)

$$D_0 \equiv 131072$$

➤ 参数说明

- $D(H)$ 是本区块的难度，由基础部分 $P(H)_{H_d} + x \times \varsigma_2$ 和难度炸弹部分 ϵ 相加得到。
- $P(H)_{H_d}$ 为父区块的难度，每个区块的难度都是在父区块难度的基础上进行调整。
- $x \times \varsigma_2$ 用于自适应调节出块难度，维持稳定的出块速度。
- 基础部分有下界，为最小值 $D_0 = 131072$ 。

1. 难度炸弹是为了向权益证明过渡
2. 红色部分设置的是挖矿的难度下限

自适应难度调整 $x \times \varsigma_2$

$$(43) \quad x \equiv \left\lfloor \frac{P(H)_{H_d}}{2048} \right\rfloor$$

$$(44) \quad \varsigma_2 \equiv \max \left(y - \left\lfloor \frac{H_s - P(H)_{H_s}}{9} \right\rfloor, -99 \right)$$

➤ 参数说明

- x 是调整的单位， ς_2 为调整的系数。
- y 和父区块的uncle数有关。如果父区块中包括了uncle ,则 y 为2, 否则 y 为1。
 - 父块包含uncle时难度会大一个单位，因为包含uncle时新发行的货币量大，需要适当提高难度以保持货币发行量稳定。
- 难度降低的上界设置为 -99 ，主要是应对被黑客攻击或其他目前想不到的黑天鹅事件。

$$y - \left\lfloor \frac{H_s - P(H)_{H_s}}{9} \right\rfloor$$

➤ 参数说明

- H_s 是本区块的时间戳， $P(H)_{H_s}$ 是父区块的时间戳，均以秒为单位，并规定 $H_s > P(H)_{H_s}$ 。
 - 该部分是稳定出块速度的最重要部分：出块时间过短则调大难度，出块时间过长则调小难度。

➤ 以父块不带uncle($y = 1$)示例

- 出块时间在 $[1,8]$ 之间，出块时间过短，难度调大一个单位。
- 出块时间在 $[9,17]$ 之间，出块时间可以接受，难度保持不变。
- 出块时间在 $[18,26]$ 之间，出块时间过长，难度调小一个单位。
-

难度炸弹 ϵ

$$\epsilon \equiv \left\lfloor 2^{\lfloor H'_i \div 100000 \rfloor - 2} \right\rfloor$$
$$H'_i \equiv \max(H_i - 3000000, 0)$$

➤为什么设置难度炸弹？

- 设置难度炸弹的原因是要降低迁移到PoS协议时发生fork的风险：到时挖矿难度非常大，所以矿工有意愿迁移到PoS协议。

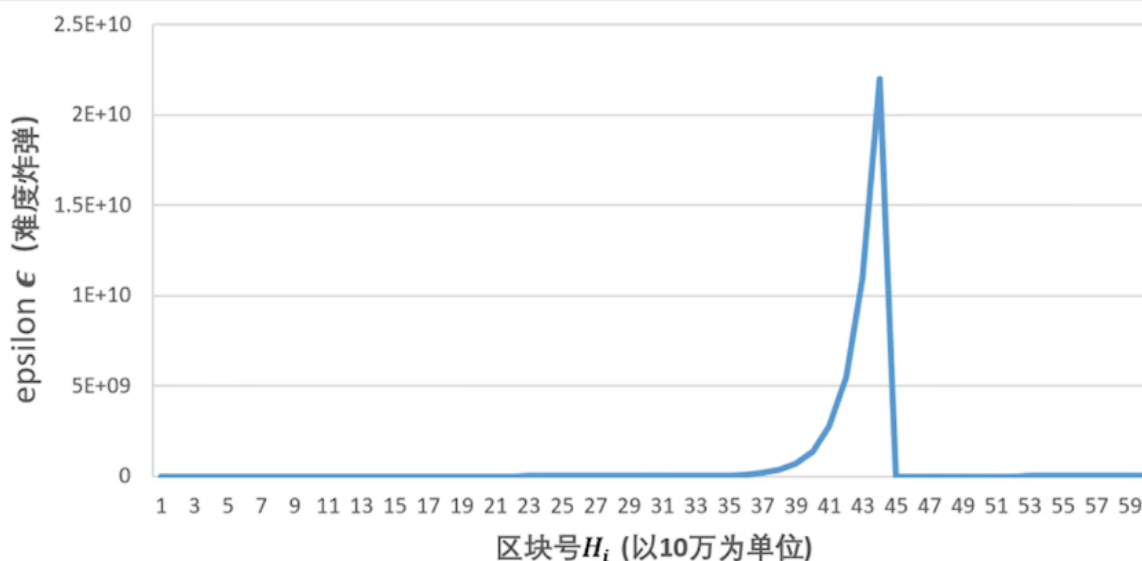
➤参数说明

- ϵ 是2的指数函数，每十万个块扩大一倍，后期增长非常快，这就是难度“炸弹”的由来。
- H'_i 称为fake block number，由真正的block number H_i 减少三百万得到。这样做的原因是低估了PoS协议的开发难度，需要延长大概一年半的时间(EIP100)。

由于权益证明会导致现有的ETH挖矿体系的硬件设备报废，为了防止出现节点因为不同的意见（造成社区的分裂）而出现分叉的情况，设计了难度炸弹。

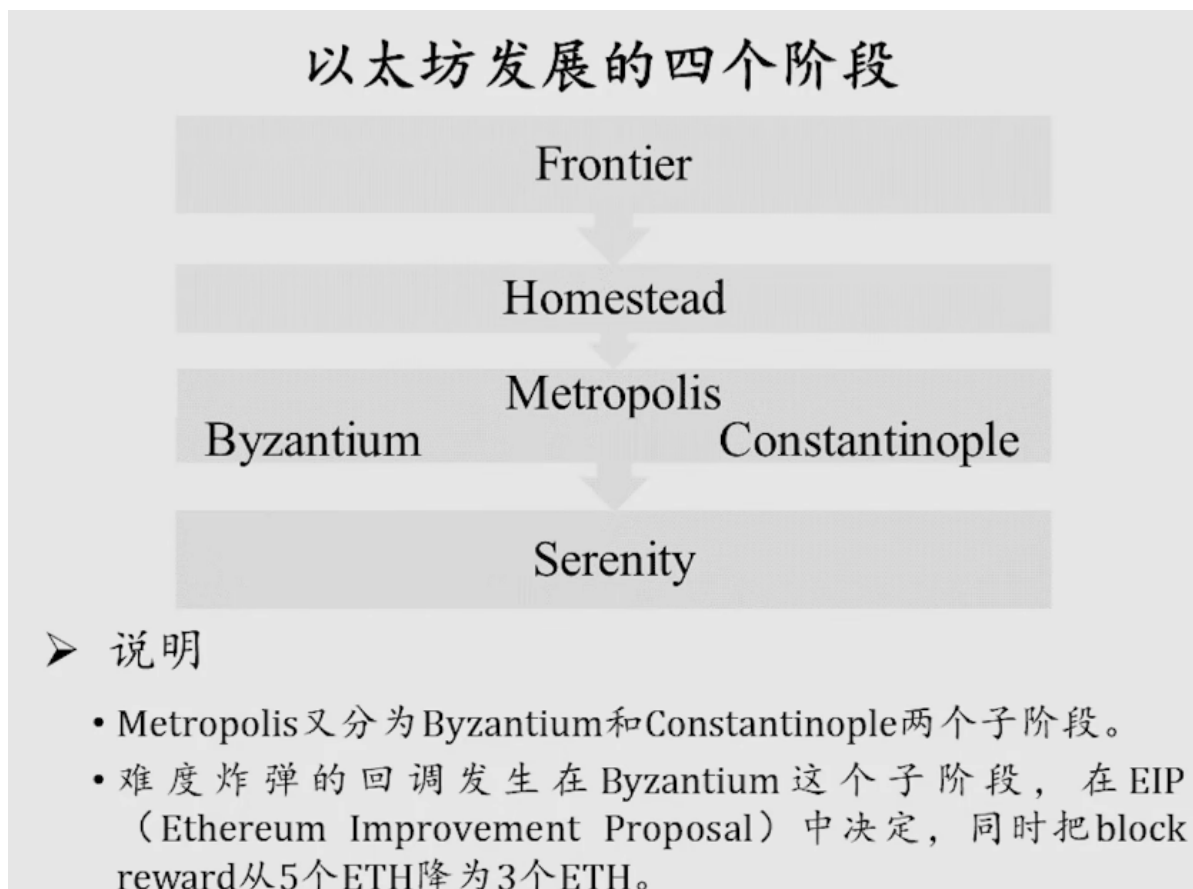
之前是设置难度炸弹逼大家转，后来是PoS设计不出来，不得不延长“爆炸”的时间，将区块号回退3000000个。

难度炸弹（difficulty bomb）的威力



前半部分，影响挖矿难度的主要是“基础调整”的公式；难度炸弹爆炸的时候，挖矿难度由它决定，剧增，使得人们转向PoS的证明共识。但是由于研究设计PoS遇到问题，不得不回调难度炸弹的区块数，又出现了此后的情况。

ETH的发展阶段



BIP (BitCoin Improvement Proposal)

在下调控矿难度（难度炸弹的回调）的同时要下调挖矿的收入，这是为了

1. 让之前较难时候挖到区块的矿工承受的住一夜之间难度骤减的心理冲击，得到一些心理安慰。
2. 同时也维持了挖出货币的总供应量

代码实现

拜占庭阶段计算挖矿难度的调整代码

```
// calcDifficultyByzantium is the difficulty adjustment algorithm.
// It returns the difficulty that a new block should have when created at the
// given time,
// given the parent block's time and difficulty. The calculation uses the
// Byzantium rules.
func calcDifficultyByzantium(time uint64, parent *types.Header) *big.Int {
    // https://github.com/ethereum/EIPs/issues/100
    // algorithm:
    // diff = (parent diff + (parent diff / 2048 * max((2 if len(parent.uncles)
else 1) * ((timestamp - parent.timestamp) * 0.9), -99)) + 2^(periodcount-2)
    bigTime := new(big.Int).SetUint64(time)
    bigParentTime := new(big.Int).Set(parent.Time)
    x := new(big.Int)
    y := new(big.Int)
```

```
// More code goes here...  
}
```

基础部分计算

```
// calcDifficultyByzantium is the difficulty adjustment algorithm.  
// It returns the difficulty that a new block should have when created at the  
// given time,  
// given the parent block's time and difficulty. The calculation uses the  
// Byzantium rules.  
func calcDifficultyByzantium(time uint64, parent *types.Header) *big.Int {  
    // https://github.com/ethereum/EIPs/issues/100  
    // algorithm:  
    // diff = (parent diff + (parent diff / 2048 * max((2 if len(parent.uncles)  
else 1) * ((timestamp - parent.timestamp) * 0.9), -99)) + 2^(periodcount-2)  
    bigTime := new(big.Int).SetUint64(time)  
    bigParentTime := new(big.Int).Set(parent.Time)  
    x := new(big.Int)  
    y := new(big.Int)  
  
    // Calculate x: (2 if len(parent.uncles) else 1) * ((timestamp -  
parent.timestamp) / 9)  
    x.Sub(bigTime, bigParentTime)  
    x.Div(x, big.NewInt(9))  
    if parent.UncleHash == types.EmptyUncleHash {  
        x.Sub(big.NewInt(1), x)  
    } else {  
        x.Sub(big.NewInt(2), x)  
    }  
  
    //与-99相比  
    // Calculate max((2 if len(parent.uncles) else 1) * ((timestamp -  
parent.timestamp) / 9), -99)  
    if x.Cmp(big.NewInt(-99)) < 0 {  
        x.Set(big.NewInt(-99))  
    }  
  
    // Calculate y: parent difficulty / 2048  
    y.Div(parent.Difficulty, big.NewInt(2048))  
  
    // Calculate diff: parent diff + (parent diff / 2048 * max((2 if  
len(parent.uncles) else 1) * ((timestamp - parent.timestamp) / 9), -99))  
    x.Mul(y, x)  
    x.Add(parent.Difficulty, x)  
  
    // Set minimum difficulty  
    if x.Cmp(big.NewInt(131072)) < 0 {  
        x.Set(big.NewInt(131072))//难度下限  
    }  
  
    return x  
}
```

难度炸弹计算

```
// Calculate a fake block number for the ice-age delay:
// https://github.com/ethereum/EIPs/pull/669
// Fake block number = min(0, block.number - 3,000,000)
fakeBlockNumber := new(big.Int)
if parent.Number.Cmp(big.NewInt(2999999)) >= 0 { //判断的是负区块的序号
    fakeBlockNumber.Sub(parent.Number, big.NewInt(2999999))
}

// For the exponential factor:
periodcount := new(big.Int).Set(fakeBlockNumber)
periodcount.Div(periodcount, expDiffPeriod)

// The exponential factor, commonly referred to as "the bomb":
// diff = diff + 2^(periodcount - 2)
if periodcount.Cmp(big.NewInt(1)) > 0 {
    expDiffPeriod := big.NewInt(1000000)
    y := new(big.Int).Sub(periodcount, big.NewInt(2))
    y.Exp(big.NewInt(2), y, nil)
    x.Add(x, y) // x 基础部分难度 y 难度炸弹的附加难度
}
```